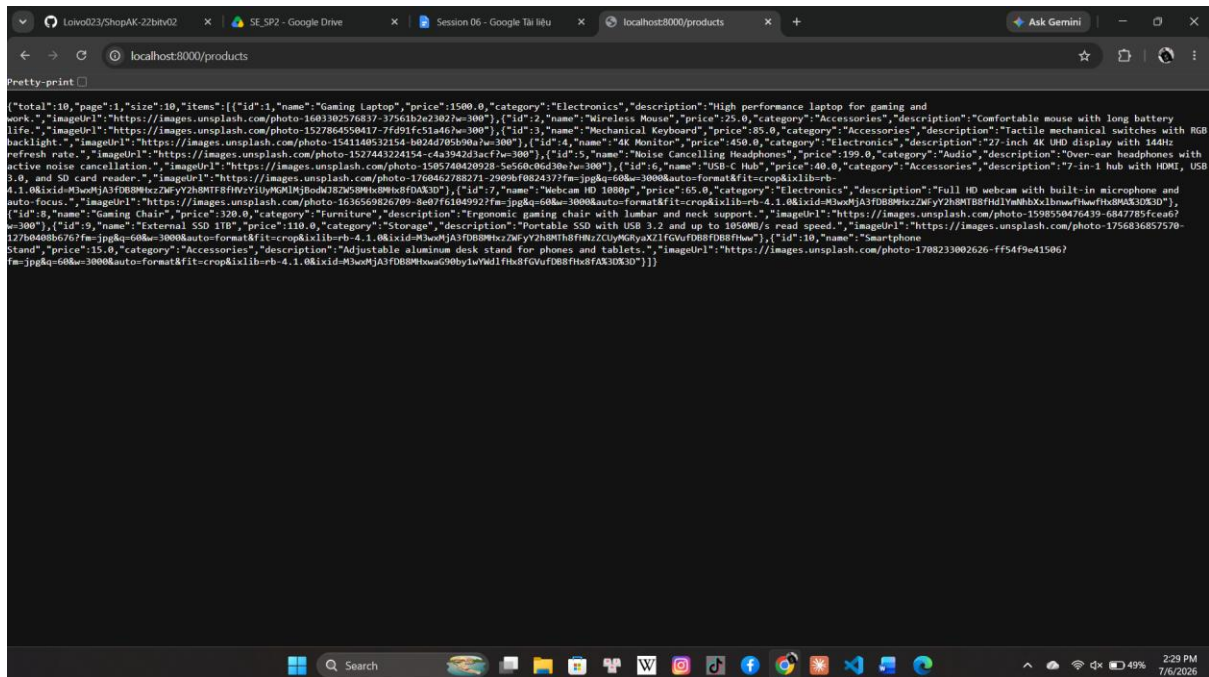


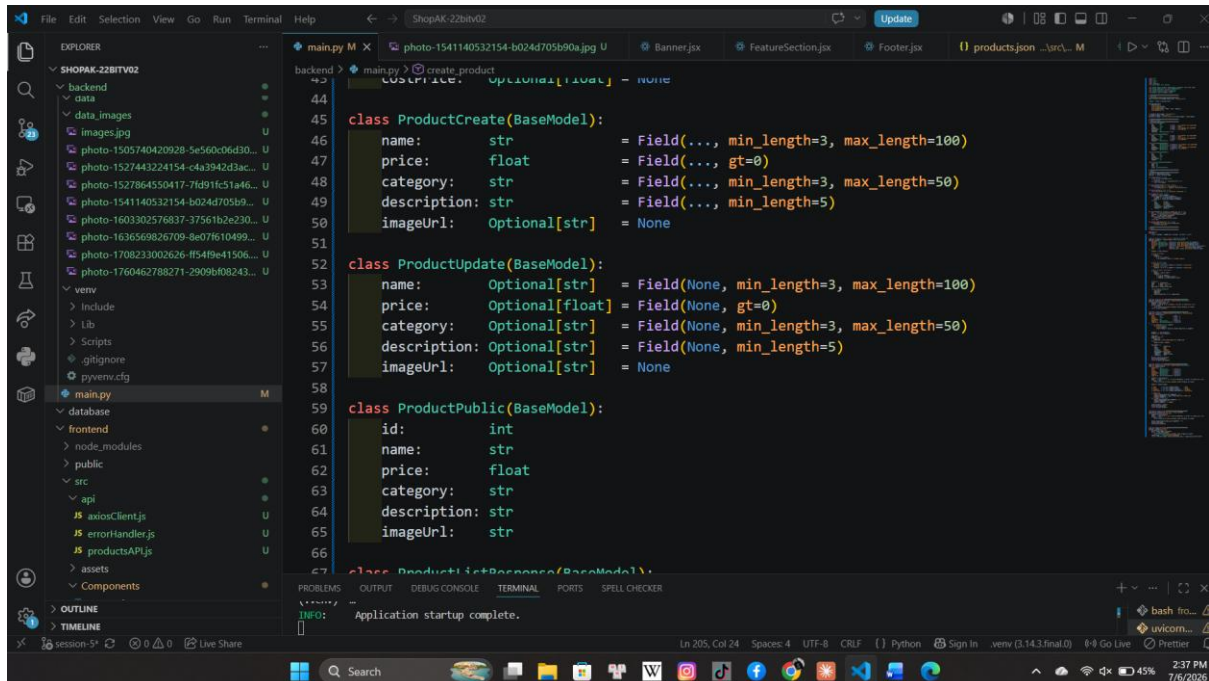
CORS + GET /products from frontend

Responde header sẽ hiển thị: Access-Control-Allow-Origin: <http://localhost:5173>

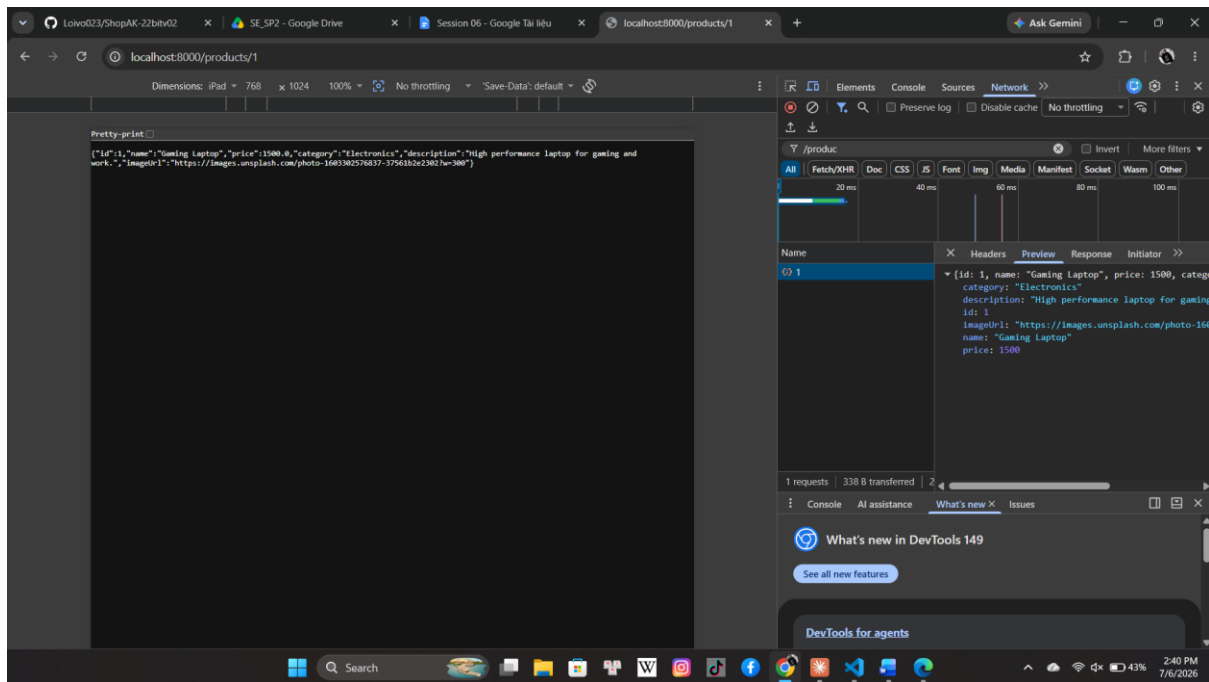


Input/output schemas + hidden costPrice

Main.py sẽ bao gồm: ProductCreate, ProductUpdate, ProductPublic classes

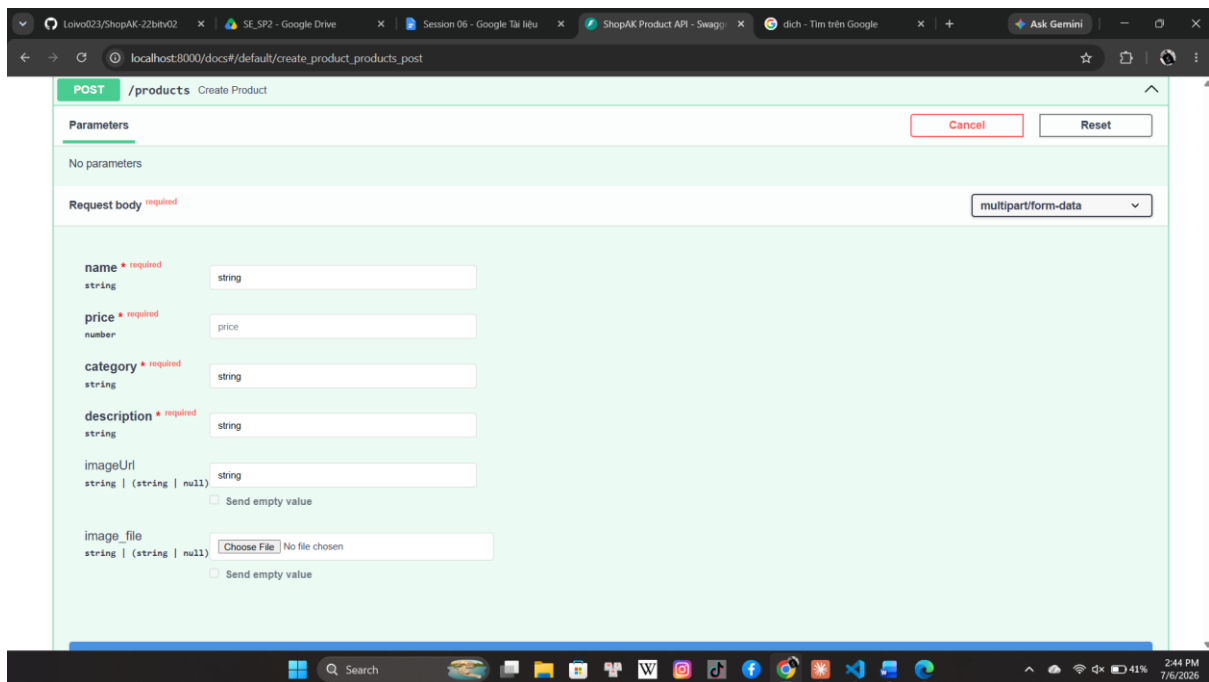


Và sau đó chúng ta vào đường liên kết: <http://localhost:8000/products/1> bấm f12 Network Response tab sẽ hiển thị JSON mà không có hiển thị mà ẩn đi costPrice



POST and PUT endpoints

FastAPI Swagger UI tại đường liên kết: <http://localhost:8000/docs> hiển thị POST /products biểu mẫu đã được mở rộng với đầy đủ các trường, bao gồm cả tính năng tải ảnh lên.



Tương tự với PUT /products/{id}

Swagger UI interface for the `update_product` endpoint. The form contains the following fields:

- name**: string | (string | null) ☐ Send empty value
- price**: number | (number | null) ☐ Send empty value
- category**: string | (string | null) ☐ Send empty value
- description**: string | (string | null) ☐ Send empty value
- imageUrl**: string | (string | null) ☐ Send empty value
- image_file**: string | (string | null) No file chosen ☒ Send empty value

Execute

Responses

Sau khi submitting POST, Phản hồi hiển thị sản phẩm mới được tạo

Swagger UI interface showing the response for the `POST` request to `http://localhost:8000/products`.

Request URL: `http://localhost:8000/products`

Server response

Code: 201

Response body

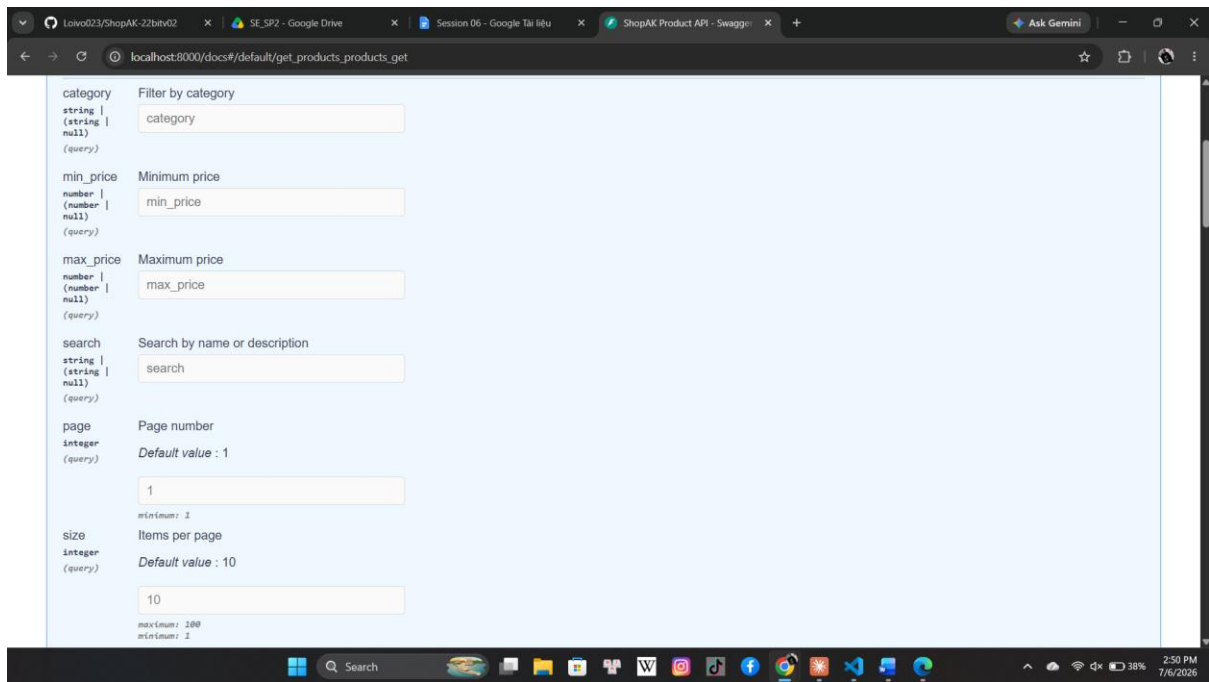
```
{
  "id": 11,
  "name": "string",
  "price": 10000,
  "category": "string",
  "description": "string",
  "imageUrl": "http://localhost:8000/images/11_285a94e5.png"
}
```

Response headers

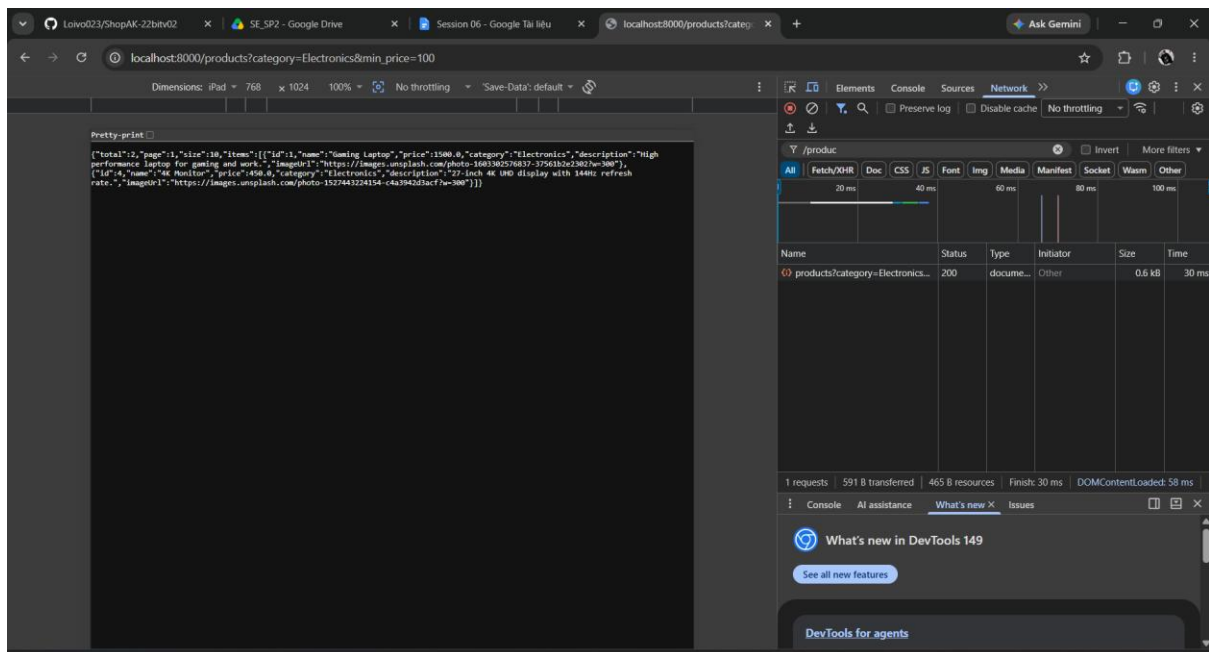
```
access-control-allow-credentials: true
content-length: 142
content-type: application/json
date: Mon, 06 Jul 2026 07:47:32 GMT
server: uvicorn
```

GET /products with query parameters

Swagger UI hiển thị GET /products được mở rộng với tất cả các tham số truy vấn (category, min_price, max_price, page, size)

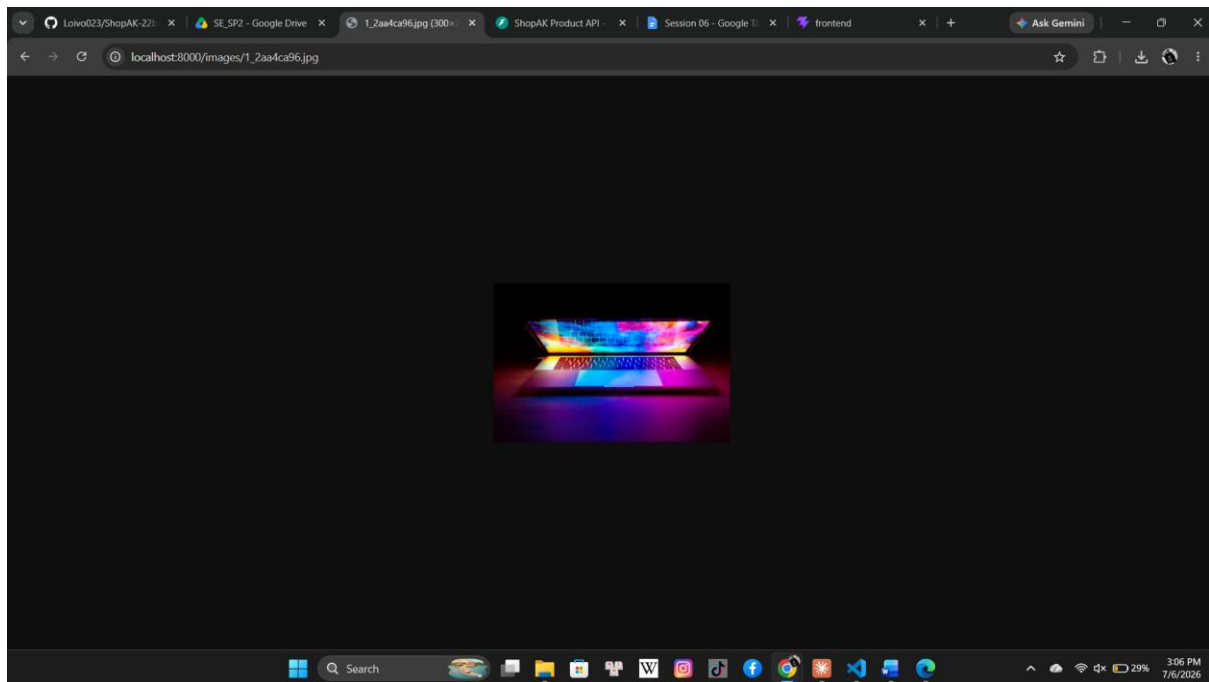


Nếu chúng ta nhập đường dẫn URL hiển thị một yêu cầu đã được lọc, chẳng hạn như http://localhost:8000/products?category=Electronics&min_price=100 ta có thể thấy chi tiết json trong network tab

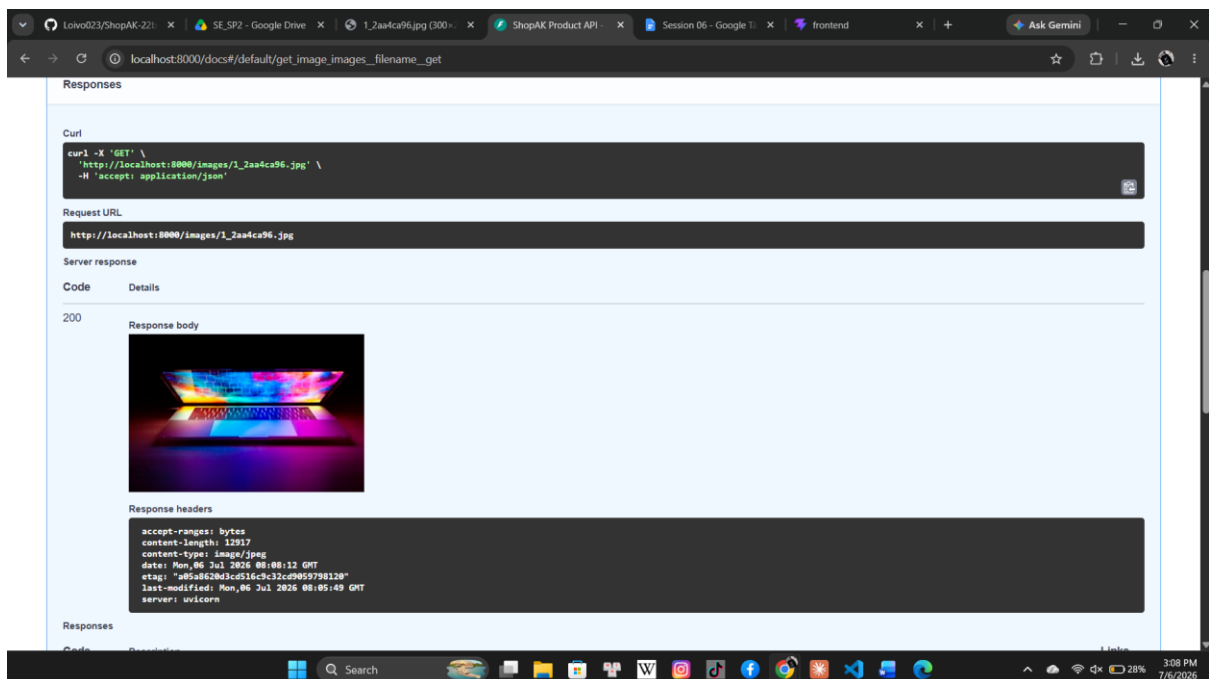


GET /images/{filename}

Chúng ta mở trình duyệt trực tiếp tại http://localhost:8000/images/1_2aa4ca96.jpg và cho chúng ta kết quả hiển thị hình ảnh



Swagger UI hiển thị endpoint GET /images/{filename}



Validation errors

Swagger UI chúng ta sẽ thử gửi yêu cầu POST /products mà không có image và imageUrl

loivo023/ShopAK-22: x SE_SP2 - Google Drive x 1_2aa4ca96.jpg (300 x 300) x ShopAK Product API - Swagger x Session 06 - Google Tài liệu x frontend x Ask Gemini x

localhost:8000/docs#/default/create_product_products_post

No parameters

Request body **required** multipart/form-data

name **required**
string

price **required**
number

category **required**
string

description **required**
string

imageUrl
string | (string | null)
☐ Send empty value

image_file
string | (string | null)
☐ Send empty value

Responses

màn hình sẽ phản hồi lỗi 422.

loivo023/ShopAK-22bitv02 x SE_SP2 - Google Drive x ShopAK Product API - Swagger x Session 06 - Google Tài liệu x frontend x Ask Gemini x

localhost:8000/docs#/default/create_product_products_post

Curl

```
curl -X 'POST' \
  'http://localhost:8000/products' \
  -H 'accept: application/json' \
  -H 'Content-Type: multipart/form-data' \
  -F 'name=test' \
  -F 'price=10' \
  -F 'category=test' \
  -F 'description=test'
```

Request URL

http://localhost:8000/products

Server response

Code	Details
422	Error: Unprocessable Content Response body <pre>{ "detail": "Image is required: provide image_file or imageUrl." }</pre> Response headers <pre>access-control-allow-credentials: true content-length: 63 content-type: application/json date: Mon, 06 Jul 2025 08:14:16 GMT server: uvicorn</pre>

Responses

Code	Description	Links
201	Successful Response	No links

Media type: application/json

Giờ chúng ta gửi giá trị price là -1 hoặc 0

No parameters

Request body required multipart/form-data

name * required
string

price * required
number

category * required
string

description * required
string

imageUrl
string | (string | null)
☐ Send empty value

image_file
string | (string | null) No file chosen
☐ Send empty value

Execute Clear

Responses

phần nội dung phản hồi (response body) của lỗi 422.

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8000/products' \
  -H 'accept: application/json' \
  -H 'Content-Type: multipart/form-data' \
  -F 'name=A' \
  -F 'price=-1' \
  -F 'category=test' \
  -F 'description=test' \
  -F 'imageUrl=http://img'
```

Request URL

<http://localhost:8000/products>

Server response

Code	Details
422	Error: Unprocessable Content

Response body

```
{
  "detail": "Price must be greater than 0."
}
```

Response headers

```
access-control-allow-credentials: true
content-length: 42
content-type: application/json
date: Mon, 06 Jul 2026 08:27:48 GMT
server: uvicorn
```

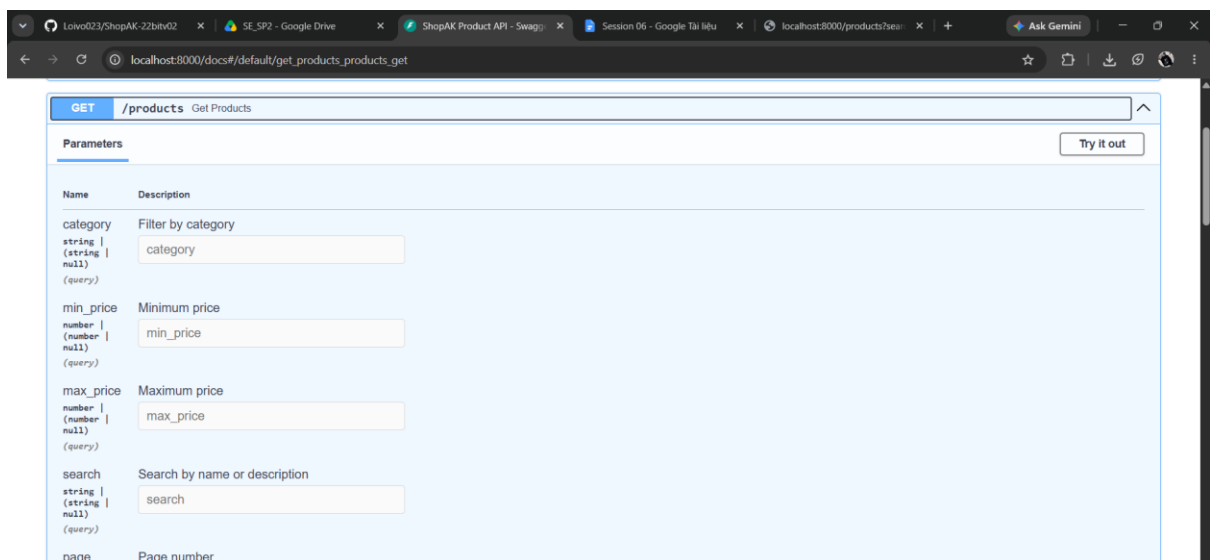
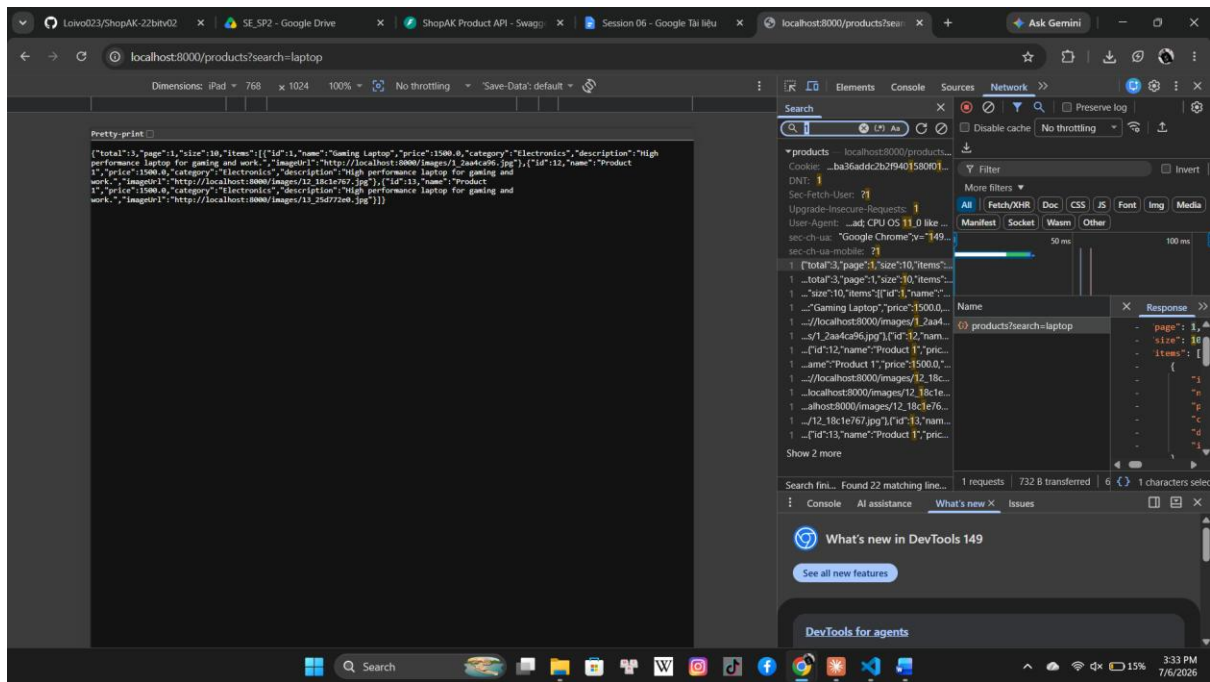
Responses

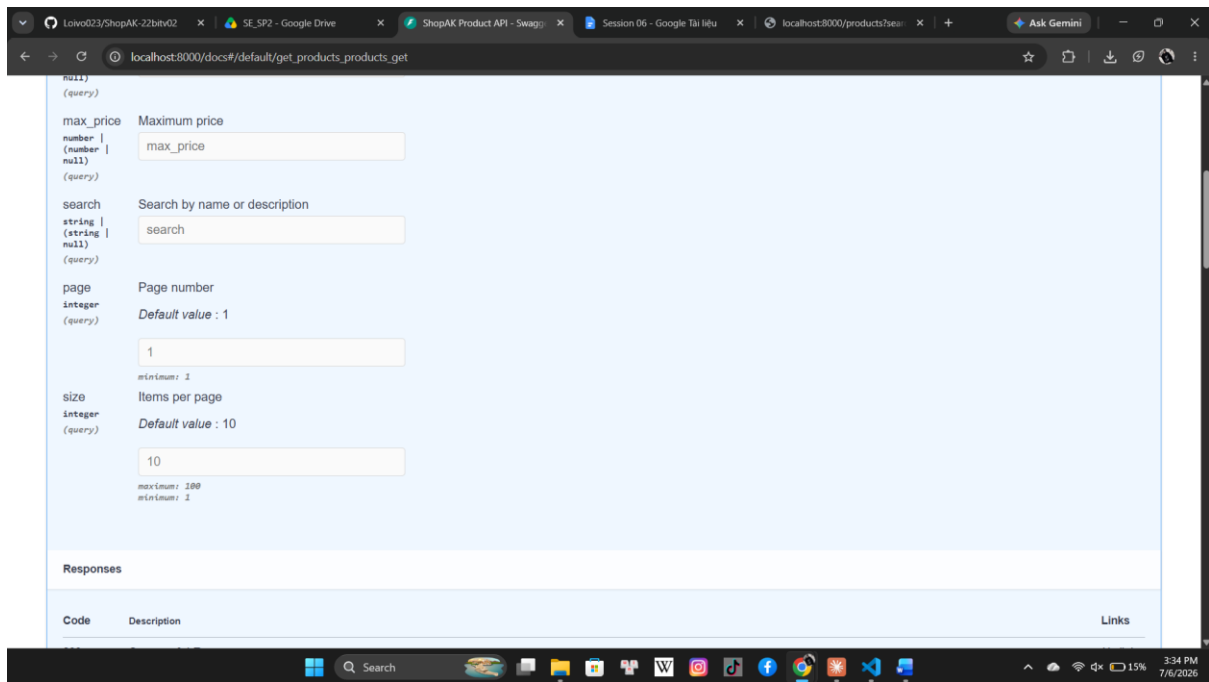
Code	Description
------	-------------

Links

Search query parameter

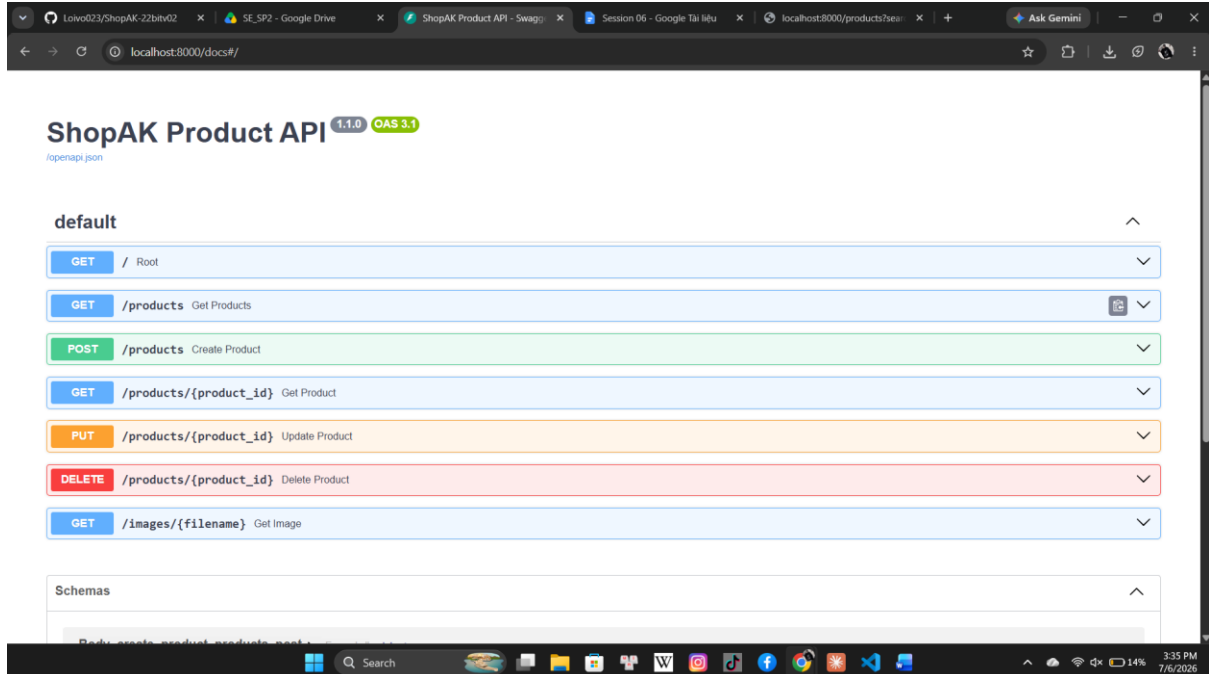
Chúng ta nhập URL trên trình duyệt: <http://localhost:8000/products?search=laptop>





Swagger UI hiển thị tham số tìm kiếm trong yêu cầu GET /products

Summary



1. Tại sao tách schema input và output lại hữu ích?

Trong dự án ShopAK, chúng tôi sử dụng ba schema riêng biệt: ProductCreate cho đầu vào khi tạo sản phẩm, ProductUpdate cho đầu vào khi cập nhật, và ProductPublic cho đầu ra trả về client. Việc tách biệt này mang lại nhiều lợi ích quan trọng.

Thứ nhất, nó giúp **bảo mật dữ liệu nhạy cảm** các trường như costPrice (giá vốn) và imagePath (đường dẫn nội bộ) chỉ tồn tại trong model nội bộ và không bao giờ bị lộ ra ngoài API. Client chỉ nhận được những gì họ cần thông qua ProductPublic.

Thứ hai, schema input và output có thể **thay đổi độc lập** với nhau ví dụ có thể thêm trường mới vào output mà không ảnh hưởng đến cách client gửi dữ liệu lên.

Thứ ba, FastAPI sử dụng response_model để **tự động lọc và validate** dữ liệu trả về, giúp API luôn trả về đúng cấu trúc đã định nghĩa.

2. CORS, file upload và pagination làm cho API "thực tế" hơn như thế nào?

CORS (Cross Origin Resource Sharing) cho phép frontend React chạy trên localhost:5173 gọi đến backend FastAPI trên localhost:8000 mà không bị trình duyệt chặn. Đây là yêu cầu bắt buộc trong mọi hệ thống web có frontend và backend chạy riêng biệt, phản ánh đúng kiến trúc thực tế trong môi trường production.

File upload cho phép người dùng đăng tải hình ảnh sản phẩm trực tiếp lên server thay vì chỉ dùng URL bên ngoài. Backend lưu file vào thư mục data_images/ và phục vụ lại qua endpoint /images/{filename}, mô phỏng cách hoạt động của các hệ thống lưu trữ file thực tế.

Pagination (phân trang) với các tham số page và size giúp API không trả về toàn bộ dữ liệu trong một lần gọi. Điều này rất quan trọng khi số lượng sản phẩm lớn, giúp giảm tải cho server, tiết kiệm băng thông và cải thiện tốc độ phản hồi đây là tiêu chuẩn phổ biến trong các API thương mại điện tử thực tế.